

Chapter 2: Classical Exact Algorithms for the Capacitated Vehicle Routing Problem

Vehicle Routing: Problems, Methods, and Applications
Toth & Vigo (eds.), 2nd Edition, SIAM 2014

Frédéric Semet Paolo Toth Daniele Vigo

Slides prepared for self-study

Chapter Outline I

This chapter surveys the most important exact methods developed for the Capacitated Vehicle Routing Problem (CVRP), tracing progress from the first Branch-and-Bound algorithms in the late 1960s through the state-of-the-art Branch-and-Cut procedures of the 2000s.

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms

Chapter Outline II

- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

1 Introduction: Why Exact Methods Matter

2 Branch-and-Bound Algorithms

3 Early Set Partitioning Algorithms

4 Branch-and-Cut Algorithms

5 Families of Cuts

6 Separation Procedures

7 Comparison of B&C Methods

8 Conclusions and Future Directions

9 Bibliography

10 Introduction: Why Exact Methods Matter

11 Branch-and-Bound Algorithms

12 Early Set Partitioning Algorithms

13 Branch-and-Cut Algorithms

14 Families of Cuts

15 Separation Procedures

16 Comparison of B&C Methods

17 Conclusions and Future Directions

18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms**
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms**
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms**
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts**
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures**
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods**
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions**
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography**
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter**
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

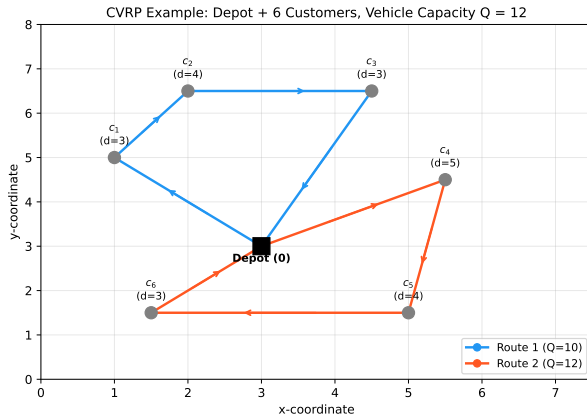
The Capacitated Vehicle Routing Problem I

Problem Definition (CVRP)

Given a depot (vertex 0) and a set $V' = \{1, \dots, n\}$ of customers embedded in a graph $G = (V, A)$ with $V = \{0\} \cup V'$, the CVRP asks for a minimum-cost set of routes, each starting and ending at the depot, such that every customer is visited exactly once and the total demand on each route does not exceed the vehicle capacity Q .

Each customer $i \in V'$ has a non-negative demand d_i . A fleet of m identical vehicles, each of capacity Q , is available at the depot. The cost of travelling from vertex i to vertex j is c_{ij} ; for most benchmark instances c_{ij} is the Euclidean distance between customers placed in the plane.

The Capacitated Vehicle Routing Problem II



Example: Six customers with demands shown. Vehicle capacity $Q = 12$.

- Route 1 (blue): depot $\rightarrow c_1 \rightarrow c_2 \rightarrow c_3 \rightarrow$ depot. Total demand $3 + 4 + 3 = 10 \leq 12$. ✓
- Route 2 (orange): depot $\rightarrow c_4 \rightarrow c_5 \rightarrow c_6 \rightarrow$ depot. Total demand $5 + 4 + 3 = 12 \leq 12$. ✓

The objective is to minimise the total travel cost (sum of arc costs) over all routes.

The Capacitated Vehicle Routing Problem III

CVRP as a Mixed-Integer Program

Introduce binary variable $x_{ij} \in \{0, 1\}$ equal to 1 if a vehicle travels directly from vertex i to vertex j , and 0 otherwise. Let $x(\delta(S)) = \sum_{(i,j) \in \delta(S)} x_{ij}$ denote the number of arcs leaving subset $S \subseteq V$.

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (P)$$

$$\text{s.t. } x(\delta(\{i\})) = 2, \quad \forall i \in V' \quad (1)$$

$$x(\delta(S)) \geq 2 \lceil d(S)/Q \rceil, \quad \forall S \subseteq V', S \neq \emptyset \quad (2)$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i,j) \in A \quad (3)$$

Constraint (1) ensures that every customer is visited exactly once (degree-2 condition). Constraint (2) is the *rounded capacity inequality* (RCI): it guarantees that at least $\lceil d(S)/Q \rceil$ vehicles service the customer set S , so at least $2\lceil d(S)/Q \rceil$ arcs must cross the boundary $\delta(S)$. The RCI simultaneously eliminates sub-tours and enforces capacity feasibility.

The Capacitated Vehicle Routing Problem IV

Key Insight: Why Exact Methods Are Difficult

The CVRP is NP-hard in the strong sense: no polynomial-time algorithm is known, and decades of research suggest none exists. However, exact algorithms are *indispensable* because:

- 1 They provide **certified optimal solutions** that validate the quality of heuristics.
- 2 They supply **tight lower bounds** used to assess the optimality gap of practical solutions.
- 3 Modern B&C can now solve instances with up to 135–200 customers to proven optimality, covering many real-world scenarios.

The chapter is organised in three sections: Section 2.2 covers Branch-and-Bound; Section 2.3 covers Set Partitioning formulations solved by column generation; Section 2.4 covers Branch-and-Cut, the dominant approach today.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms**
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

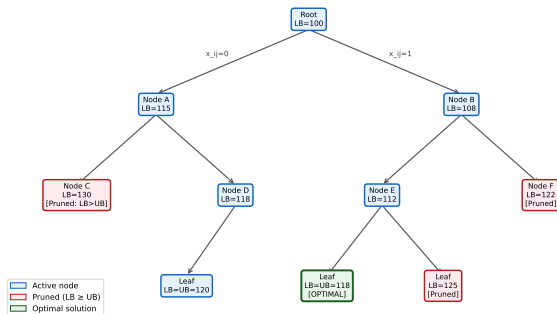
Branch-and-Bound: Core Idea I

Branch-and-Bound (B&B) solves a combinatorial optimisation problem by *implicitly* enumerating all feasible solutions. The key insight is that large portions of the search space can be **pruned**—discarded without explicit examination—whenever a *lower bound* on the cost of any solution reachable from the current node is at least as large as the cost of the best solution found so far (the *upper bound*).

The algorithm maintains a tree of *subproblems*. Each node in the tree corresponds to a subproblem obtained by fixing some variables (e.g., setting $x_{ij} = 1$ or $x_{ij} = 0$). The root node is the full, unfixed problem. Leaf nodes are either infeasible, pruned, or correspond to a complete integer solution.

Branch-and-Bound: Core Idea II

Branch-and-Bound Search Tree for CVRP
(Blue = active, Red = pruned, Green = optimal leaf)



Three pruning criteria:

- 1 **Infeasibility:** the node's subproblem has no feasible solution.
- 2 **Bound:** the node's lower bound $LB \geq$ current upper bound UB . No better solution can lie in this subtree.
- 3 **Optimality:** the LP solution is already integer — it is a feasible CVRP solution. Update UB if it improves the incumbent.

In the figure, red nodes are pruned by criterion 2 ($LB \geq UB=120$), and the green leaf with $LB=UB=118$ is the **optimal solution**.

Branch-and-Bound: Core Idea III

Generic B&B Algorithm for CVRP

- 1: Initialise: $UB \leftarrow$ cost of a heuristic solution; open-list $\mathcal{L} \leftarrow \{\text{root node}\}$.
- 2: **while** $\mathcal{L} \neq \emptyset$ **do**
- 3: Select and remove a node N from $\mathcal{L}$.
- 4: Compute lower bound $LB(N)$ for node N .
- 5: **if** $LB(N) \geq UB$ **then** **prune** N ; **continue**.
- 6: **end if**
- 7: **if** LP solution at N is integer and feasible **then**
- 8: Update $UB \leftarrow LB(N)$; record solution; **continue**.
- 9: **end if**
- 10: **Branch**: select a fractional variable x_{ij} ; create two children by adding constraint $x_{ij} = 0$ or $x_{ij} = 1$; add both to $\mathcal{L}$.
- 11: **end while**
- 12: **return** best solution found with cost UB .

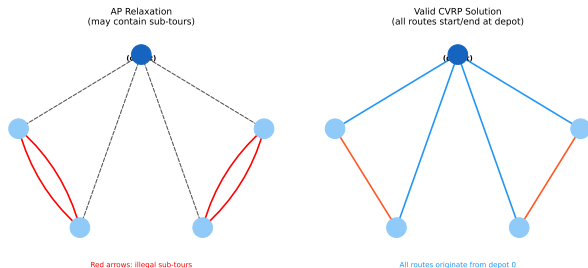
Key Insight: Lower Bounds Drive B&B Efficiency

The quality of the lower bound is the single most important factor in B&B performance. A tight LB prunes more nodes and leads to faster convergence. For the CVRP, effective lower bounds come from LP relaxations (solved once per node) and problem-specific relaxations such as the Assignment Problem (AP) and the q -routes relaxation.

2.2.1 Assignment and Matching Relaxations I

The assignment and matching-based bounds, pioneered by Christofides and Eilon (1969) and refined by Toth and Vigo, provide fast lower bounds exploiting the structure of the CVRP.

Assignment Problem (AP) relaxation. The AP is obtained from the CVRP by dropping the capacity constraints and the sub-tour elimination constraints: we merely require each vertex to have in-degree 1 and out-degree 1. The AP is solvable in $O(n^3)$ time (Hungarian algorithm) and its optimal value is a lower bound for the CVRP. However, the AP solution often contains sub-tours, as illustrated below.



Left: AP solution for 5 customers. Two sub-tours $\{1 \rightarrow 2 \rightarrow 1\}$ and $\{3 \rightarrow 4 \rightarrow 3\}$ are formed (red arrows) which are *infeasible* for CVRP.

Right: A valid CVRP solution with all routes starting/ending at the depot. The AP lower bound is used as the *root bound* z_{AP} in the B&B tree; any CVRP solution has cost $\geq z_{AP}$.

2.2.1 Assignment and Matching Relaxations II

The q -routes relaxation. Introduced by Christofides, Mingozzi, and Toth (1981), this relaxation generates a stronger lower bound by replacing each CVRP route with a shortest Hamiltonian path from the depot to vertex k using at most q edges, where k varies over all customers. It is formulated as a shortest-path problem on an acyclic graph and solved in pseudo-polynomial time. When $q = 2$ (allowing 1 intermediate customer), the bound is already substantially tighter than the AP bound.

In the B&B framework, the lower bound at each node is computed by:

- 1 Solving the AP relaxation of the *reduced* subproblem (with fixed arcs incorporated), yielding bound z_{AP} .
- 2 Optionally adding the q -routes component to tighten z_{AP} .
- 3 Using the combined bound as the node's LB.

The overall AP additive approach (Christofides and Eilon, 1969) solved problems up to $n = 50$ customers. The refinements of Fisher (1994) and Toth and Vigo (2002) extended the reach to $n = 100$.

2.2.1 Assignment and Matching Relaxations III

Worked Example: AP Relaxation for 4 Customers

Consider a CVRP with depot 0 and customers $\{1, 2, 3, 4\}$ and the following cost matrix (symmetric, $c_{ij} = c_{ji}$):

$$C = \begin{pmatrix} - & 4 & 8 & 6 & 5 \\ 4 & - & 3 & 7 & 9 \\ 8 & 3 & - & 5 & 4 \\ 6 & 7 & 5 & - & 2 \\ 5 & 9 & 4 & 2 & - \end{pmatrix}, \quad Q = 6, \quad d_1 = 3, \quad d_2 = 3, \quad d_3 = 4, \quad d_4 = 4.$$

The AP relaxation assigns each vertex a unique successor. One AP optimal assignment might yield: $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 0, 3 \rightarrow 4, 4 \rightarrow 3$ with cost $4 + 3 + 8 + 2 + 2 = 19$. This contains the sub-tour $\{3 \rightarrow 4 \rightarrow 3\}$. Hence $z_{AP} = 19$ is a *lower bound* for the CVRP.

A valid CVRP solution: route A: $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ (cost $4 + 3 + 8 = 15$, demand $3 + 3 = 6 \leq Q$); route B: $0 \rightarrow 3 \rightarrow 4 \rightarrow 0$ (cost $6 + 2 + 5 = 13$, demand $4 + 4 = 8 > Q$). Infeasible! Try: route B': $0 \rightarrow 3 \rightarrow 0$ (cost $6 + 6 = 12$, demand 4); route C: $0 \rightarrow 4 \rightarrow 0$ (cost $5 + 5 = 10$, demand 4). Total cost $= 15 + 12 + 10 = 37$. The AP bound $z_{AP} = 19$ is far from this, showing the bound's weakness. Adding capacity penalties tightens it.

2.2.2 Bounds Based on Spanning and Additive Approaches I

Beyond the AP relaxation, several bounding strategies improve the lower bound by incorporating capacity constraints more tightly. Two important approaches are the *spanning* bounds and the *additive* approach.

Spanning tree bound. The 1-tree relaxation (used for the TSP by Held and Karp, 1971) can be adapted for the CVRP. One removes the depot, computes a minimum spanning tree of the customer graph, and then attaches the two cheapest depot edges to each customer at the “ends” of the tree. This gives a lower bound because every CVRP tour, when the depot is removed, induces a Hamiltonian path (a special spanning tree).

Lagrangian relaxation. A more powerful approach dualises the degree constraints or capacity constraints by adding them to the objective with Lagrange multipliers λ_i . For multiplier vector λ , the Lagrangian relaxation $LR(\lambda)$ provides a lower bound:

$$LB(\lambda) = \min_{x \geq 0, x \text{ feasible for relaxed problem}} \left[\sum_{(i,j)} c_{ij} x_{ij} + \sum_i \lambda_i (x(\delta(\{i\})) - 2) \right].$$

The *Lagrangian dual* $\max_{\lambda} LB(\lambda)$ is solved by subgradient optimisation. At each step the multipliers are updated as $\lambda_i \leftarrow \lambda_i + t_k (x(\delta(\{i\})) - 2)$, where t_k is a step size satisfying $\sum_k t_k = \infty$ and $\sum_k t_k^2 < \infty$.

2.2.2 Bounds Based on Spanning and Additive Approaches II

Additive approach. The additive bounding procedure, described by Toth and Vigo (2002), combines several relaxations additively. Let z_1 be the AP lower bound. One then computes corrections:

$$z_{\text{add}} = z_1 + z_2 + \cdots + z_k,$$

where each z_ℓ is a “correction term” obtained by solving a sub-problem that accounts for features not captured in $z_{\ell-1}$. For example, z_2 may account for the minimum cost of ensuring that at least two arcs leave the depot (i.e., at least one vehicle exists). This additive scheme gives bounds significantly tighter than the AP bound alone without the computational cost of solving the full LP.

Key Insight: Additive vs. LP Bounds

The additive AP-based bounds run in $O(n^3)$ time per node, whereas LP relaxations (using interior point or simplex) can be much slower. Early B&B algorithms (1969–2002) therefore preferred AP-based bounds. Modern B&C algorithms use LP bounds enhanced by cutting planes, which are far tighter and justify the higher computational cost.

2.2.3 Branch-and-Bound Branching Strategies I

The choice of which variable to branch on, and the order in which nodes are explored, profoundly affect B&B performance. For the CVRP, several branching rules have been proposed.

Branching on arc variables. The most natural rule selects a fractional arc variable x_{ij} and creates two child nodes: one with $x_{ij} = 1$ (the arc is used) and one with $x_{ij} = 0$ (the arc is forbidden). The fractional variable closest to $\frac{1}{2}$ is often chosen to maximise the difference between the two children.

Branching on the number of vehicles. Because the number of routes m is a key parameter, branching on m is effective when the LP relaxation yields a non-integer m . The two children fix $m \leq \lfloor m^* \rfloor$ and $m \geq \lceil m^* \rceil$ respectively.

Node selection strategies:

- **Best-first:** always expand the node with the smallest LB. Minimises the number of nodes explored but may use much memory.
- **Depth-first:** expand the most recently created node (stack). Finds integer solutions quickly, uses less memory, but may explore poor subtrees.

2.2.3 Branch-and-Bound Branching Strategies II

- **Best-first with depth-first dives:** a hybrid that dives depth-first from the best node, then backtracks.

2.2.3 Branch-and-Bound Branching Strategies III

Dominance and reduction rules. Several CVRP-specific reductions can be applied at each node to simplify the subproblem:

- ① **Arc elimination:** if c_{ij} is too large (i.e., the reduced cost of x_{ij} exceeds $UB - LB$), set $x_{ij} = 0$.
- ② **Forced arcs:** if a vertex i has all but one incident arc eliminated, force the remaining arc.
- ③ **Capacity pruning:** if fixing $x_{ij} = 1$ would make any route demand exceed Q , set $x_{ij} = 0$.

These reductions shrink the problem at each node and significantly reduce the branching factor.

Historical Context

The B&B framework for CVRP was first proposed by Christofides and Eilon (1969) and solved instances up to $n = 50$. Subsequent improvements by Laporte, Mercure, and Nobert (1986) used a strengthened bounding procedure and reached $n = 100$. Toth and Vigo (2002) further refined the approach. All these algorithms were eventually superseded by B&C methods, which exploit cutting planes for much tighter bounds.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms**
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

2.3 Set Partitioning Formulation I

An alternative to the arc-variable formulation is the *Set Partitioning* (SP) formulation, which works with *route variables* rather than arc variables. This formulation was introduced by Balinski and Quandt (1964) for the TSP and adapted to the CVRP by Bramel and Simchi-Levi (1997) and others.

Idea. Enumerate (or generate implicitly) all feasible routes $r \in \mathcal{R}$. Each route r has a cost c_r and a binary parameter $a_{ir} \in \{0, 1\}$ equal to 1 if customer i is on route r . Choose a minimum-cost subset of routes that covers every customer exactly once.

Set Partitioning (SP) Model

$$\min \sum_{r \in \mathcal{R}} c_r y_r \quad (\text{SP})$$

$$\text{s.t.} \quad \sum_{r \in \mathcal{R}} a_{ir} y_r = 1, \quad \forall i \in V' \quad (4)$$

$$y_r \in \{0, 1\}, \quad \forall r \in \mathcal{R} \quad (5)$$

2.3 Set Partitioning Formulation II

Here $y_r = 1$ means route r is selected. Constraint (4) ensures each customer is served by exactly one route. Capacity constraints are embedded in $\mathcal{R}$ (only feasible routes are included). The formulation is exact but has an exponential number of columns $|\mathcal{R}|$.

2.3 Set Partitioning Formulation III

Set Partitioning Matrix: Routes vs. Customers
(Each customer must be covered by exactly one route)

Customer	r1 c=18.0	r2 c=22.5	r3 c=12.0	r4 c=20.0	r5 c=19.5	r6 c=15.0
c1	1	0	0	1	0	0
c2	1	0	0	0	1	0
c3	0	1	0	1	0	0
c4	0	1	0	0	0	1
c5	0	0	1	0	1	0

Green=1 means the route covers that customer.
SP selects a minimum-cost subset of routes that covers each customer exactly once.

Set Partitioning Matrix for 5 customers and 6 candidate routes. Each column is a route; each row is a customer. A cell is 1 (green) if the route visits that customer.

The SP problem selects a minimum-cost subset of columns such that each row has exactly one selected entry. In this example, selecting routes r_1 , r_2 , r_3 (total cost $18 + 22.5 + 12 = 52.5$) covers all 5 customers exactly once.

2.3 Set Partitioning Formulation IV

Worked Example: SP Feasibility Check

Suppose we have 4 customers with demands $d_1 = 3, d_2 = 4, d_3 = 3, d_4 = 5$ and capacity $Q = 8$. The candidate routes and their coverage are:

Route	Customers	Total Demand	Cost
r_1	$\{1, 2\}$	$7 \leq 8$	12
r_2	$\{3, 4\}$	$8 \leq 8$	15
r_3	$\{1, 3\}$	$6 \leq 8$	11
r_4	$\{2, 4\}$	$9 > 8$	— (infeasible)
r_5	$\{1, 2, 3\}$	$10 > 8$	— (infeasible)
r_6	$\{4\}$	$5 \leq 8$	9

Feasible SP solutions: $\{r_1, r_2\}$ (cost 27), $\{r_3, r_4\}$ (infeasible — r_4 dropped), $\{r_3, r_6, r_2^*\}$ where r_2^* covers $\{3, 4\}$ minus already-served $\{3\}$. The optimal is $\{r_1, r_2\}$ with cost 27.

Column Generation for SP LP Relaxation I

Since $|\mathcal{R}|$ is exponential, the LP relaxation of SP cannot be solved by simply loading all columns. Instead, the *column generation* technique is used: start with a small *restricted* set of routes (the *Restricted Master Problem*, RMP), solve the LP, then iteratively add routes (columns) that improve the objective.

How column generation works. At each iteration:

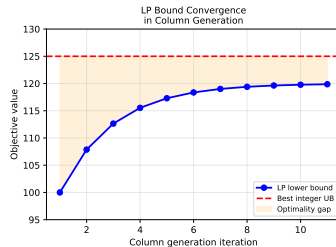
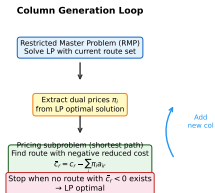
- 1 Solve the LP relaxation of the RMP. Let π_i be the optimal dual variable (shadow price) for customer i 's covering constraint.
- 2 Compute the *reduced cost* of each route $r \notin \text{RMP}$:

$$\bar{c}_r = c_r - \sum_{i \in V'} \pi_i a_{ir}.$$

A route improves the LP objective if and only if $\bar{c}_r < 0$.

- 3 Find the route with minimum reduced cost (the *pricing subproblem*). If $\bar{c}_r \geq 0$ for all routes, the current LP solution is optimal for the full SP LP.
- 4 Otherwise, add the most negative-reduced-cost route to the RMP and repeat.

Column Generation for SP LP Relaxation II



Left: the column generation loop. The dual prices π_i from the RMP guide the pricing subproblem: find the cheapest route considering these shadow prices.

Right: convergence of the LP lower bound. After 10–15 column generation iterations the bound stabilises near the optimum. The orange gap between the LP bound and the integer UB is the *optimality gap*.

The pricing subproblem for CVRP. Finding the route with minimum reduced cost is equivalent to finding a minimum-cost elementary path (visiting each customer at most once) from the depot back to the depot, with total demand $\leq Q$. This is a *resource-constrained shortest path problem* (ESPPRC), which is itself NP-hard in general. In practice:

- Dynamic programming on a state space (i, S, d) , where i is the current customer, S is the set of visited customers, and d is the accumulated demand.
- *Non-elementary relaxation*: allow repeated customers (easier to solve), giving routes that may visit a customer multiple times. Used in early implementations (Agarwal et al., 1989).
- *ng-routes* (Baldacci, Mingozzi, Roberti, 2011): a compromise allowing bounded non-elementarity, widely used in modern B&C.

Column Generation for SP LP Relaxation IV

Historical SP Algorithms for CVRP

- **Agarwal, Mathur, and Salikin (1989)**: first practical SP algorithm for CVRP. Used column generation without branch-and-bound. Solved instances up to $n = 45$.
- **Desrochers, Desrosiers, and Solomon (1992)**: rigorous column generation with ESPPRC pricing. Extended to VRPTW.
- **Fisher and Jaikumar (1981)**: generalised assignment heuristic that inspired later exact SP algorithms.

Key Insight: Column Generation Provides LP Bound

The LP relaxation of SP solved by column generation gives a lower bound at least as tight as the LP relaxation of the arc-variable formulation, and often much tighter. This makes SP-based B&C methods highly competitive.

Section Outline

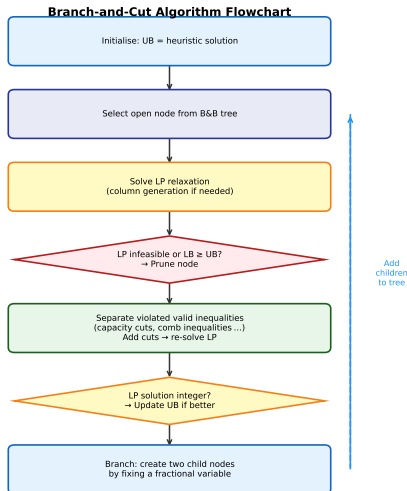
- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms**
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

2.4 Branch-and-Cut: The Dominant Approach I

Branch-and-Cut (B&C) is today the most powerful exact approach for combinatorial optimisation problems with a rich polyhedral structure, including the CVRP. The method was developed for the TSP by Padberg and Rinaldi (1991) and extended to the CVRP by Padberg and Rinaldi (1991), Laporte et al. (1986), and most influentially by Augerat et al. (1995) and Lysgaard et al. (2004).

Core idea. B&C combines the B&B tree search with an LP relaxation that is *progressively tightened* at each node by adding cutting planes (*valid inequalities*). A valid inequality is a linear constraint satisfied by *all* integer feasible CVRP solutions but violated by the current fractional LP solution. Adding it “cuts off” the fractional point, tightening the LP relaxation and raising the lower bound.

2.4 Branch-and-Cut: The Dominant Approach II



B&C Algorithm Overview:

- 1 **Initialise** with a heuristic upper bound UB .
- 2 **Select** an open node from the B&B tree.
- 3 **Solve LP relaxation** at the node (reusing cuts from the parent node).
- 4 **Prune** if LP infeasible or $LB \geq UB$.
- 5 **Separate**: find violated valid inequalities; add them to the LP and re-solve. Repeat until no violated inequality is found or a limit is reached.
- 6 If LP solution is **integer**: update UB if better.
- 7 Otherwise, **branch** on a fractional variable.
- 8 **Add children** to the tree; repeat.

2.4 Branch-and-Cut: The Dominant Approach III

Why B&C Dominates B&B for CVRP

Pure B&B relies on LP relaxations that may be very weak (large gap between LP bound and integer optimum). B&C tightens the LP relaxation by adding cutting planes, so:

- Fewer B&B nodes need to be explored.
- The LP bound at the root node may already be very close to the optimum, sometimes proving optimality without branching.
- State-of-the-art B&C (Lysgaard et al., 2004) solves instances up to 135 customers routinely.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts**
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

2.4.1 Families of Valid Inequalities I

A central component of any B&C algorithm for the CVRP is a rich library of *valid inequalities*. These inequalities are satisfied by all integer feasible CVRP solutions but may be violated by LP solutions. Adding them tightens the LP, raises the lower bound, and facilitates pruning.

In this section we survey the most important families, following the presentation of Augerat et al. (1995), Lysgaard et al. (2004), and Toth and Vigo (2002).

2.4.1 Families of Valid Inequalities II

Notation (recap)

- $V = \{0, 1, \dots, n\}$: vertex set; 0 is depot; $V' = V \setminus \{0\}$.
- $x_{ij} \in \{0, 1\}$: arc variable.
- $x(E') = \sum_{(i,j) \in E'} x_{ij}$: sum over an edge set E' .
- $\delta(S)$: set of edges with exactly one endpoint in S .
- $E(S)$: set of edges with both endpoints in S .
- $d(S) = \sum_{i \in S} d_i$: total demand of S .
- $r(S) = \lceil d(S)/Q \rceil$: minimum number of vehicles needed for S .

2.4.1 Families of Valid Inequalities III

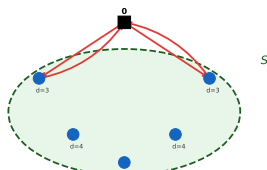
Rounded Capacity Inequalities (RCI)

For every non-empty subset $S \subseteq V'$:

$$x(\delta(S)) \geq 2r(S) = 2 \left\lceil \frac{d(S)}{Q} \right\rceil.$$

The RCI says: since at least $r(S)$ vehicles must enter and leave S , at least $2r(S)$ arcs must cross the boundary $\delta(S)$. This is the most fundamental class of valid inequalities for the CVRP; the arc-variable model (P) already contains these as constraints, but in the LP relaxation they are often violated, allowing fractional solutions.

Cut edges $\delta(S)$ for subset S



$$d(S) = \sum_{i \in S} d_i = 17, Q = 10$$

RCI: $x(\delta(S)) \geq 2 \lceil 17/10 \rceil = 4$

Rounded Capacity Inequality (RCI)

For every subset $S \subseteq V \setminus \{0\}$:

$$x(\delta(S)) \geq 2 \left\lceil \frac{d(S)}{Q} \right\rceil$$

where $x(\delta(S))$ is the number of edges crossing the boundary of S ,
 $d(S) = \sum_{i \in S} d_i$ is the total demand of S , and Q is the vehicle capacity.

Intuition: if the total demand of S exceeds kQ , then at least k vehicles must enter (and leave) S , giving at least $2k$ crossing edges.

Left: subset $S = \{1, 2, 3, 4, 5\}$ with total demand $d(S) = 17$ and $Q = 10$. The RCI requires $x(\delta(S)) \geq 2 \lceil 17/10 \rceil = 4$. The depot-to- S boundary must be crossed by at least 4 arcs (two routes in, two routes out).

2.4.1 Families of Valid Inequalities IV

Generalised Capacity Inequalities (GCI)

The GCI proposed by Augerat et al. (1995) generalises the RCI by allowing a fractional multiplier. For $S \subseteq V'$:

$$x(\delta(S)) \geq 2 \left\lceil \frac{\sum_{i \in S} d_i}{Q} \right\rceil.$$

When customers have different demands, the GCI can be stronger than the RCI because the ceiling operation is applied to the *sum* of demands rather than individual demands. In practice, the RCI and GCI coincide unless fractional demands are involved, but the GCI is the standard formulation used in the literature.

The *strengthened* version of the GCI incorporates a tighter right-hand side by exploiting the fact that each vehicle can carry at most Q units: for any vertex $v \in S$, the GCI can be strengthened to:

$$x(\delta(S)) + x(\delta(S \setminus \{v\})) \geq 4r(S) - 2r(S \setminus \{v\}).$$

2.4.1 Families of Valid Inequalities V

Framed Capacity Inequalities (FCI)

Introduced by Toth and Vigo (2002), the FCI extends the RCI by considering a *frame* around subset S : a set F of vertices “surrounding” S . Formally, for $S \subseteq V'$ and $F \supseteq S$ with $F \neq \emptyset$:

$$x(\delta(S)) \geq 2r(S) + 2r(F \setminus S) - r(F).$$

The FCI is at least as strong as the RCI when $r(F \setminus S) > 0$, because the right-hand side $2r(S) + 2r(F \setminus S) - r(F) \geq 2r(S)$ whenever routing $F \setminus S$ requires additional vehicles. These inequalities are generated by a separation procedure based on solving auxiliary LP sub-problems.

2.4.1 Families of Valid Inequalities VI

Hypotour Inequalities

Proposed by Gendreau, Laporte, and Seguin (1995), the hypotour inequality uses the concept of a “hyper-tour”—a Hamiltonian cycle visiting some customers multiple times to respect capacity. For a subset $S \subseteq V'$, the hypotour inequality reads:

$$x(E(S)) \leq |S| - r(S).$$

Here $x(E(S))$ is the number of edges with both endpoints in S . The inequality says: if $r(S)$ vehicles service S , then at most $|S| - r(S)$ arcs can be *interior* to S (the remaining arcs must connect to other vertices or the depot). When $r(S) = 1$ this reduces to the standard sub-tour elimination constraint $x(E(S)) \leq |S| - 1$.

2.4.1 Families of Valid Inequalities VII

Multistar Inequalities

Proposed by Augerat et al. (1995), the multistar inequality strengthens the RCI by incorporating individual customer degree constraints. For $S \subseteq V'$ with $|S| \geq 2$:

$$x(\delta(S)) \geq 2r(S) + \max_{v \in S} (2r(S) - x(\delta(S \setminus \{v\}))).$$

The term $2r(S) - x(\delta(S \setminus \{v\}))$ measures how much more the removal of the “hardest-to-cover” vertex v from S tightens the bound. This family was found to be very effective in practice by Lysgaard et al. (2004).

2.4.1 Families of Valid Inequalities VIII

Comb Inequalities

Generalising the classic TSP comb inequalities (Chvátal, 1973) to the CVRP. A *comb* consists of a *handle* $H \subseteq V$ and an odd number $t \geq 3$ of *teeth* $T_1, \dots, T_t \subseteq V$ satisfying:

- $H \cap T_k \neq \emptyset$ for all k (each tooth intersects the handle),
- $T_k \setminus H \neq \emptyset$ for all k (each tooth extends outside H),
- $T_k \cap T_\ell = \emptyset$ for $k \neq \ell$ (teeth are disjoint).

The comb inequality is:

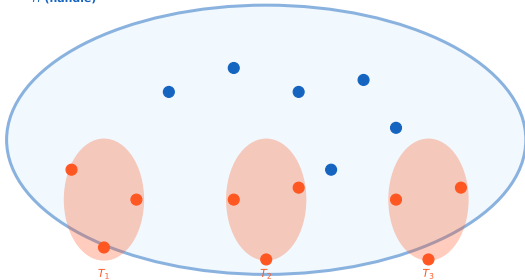
$$x(\delta(H)) + \sum_{k=1}^t x(\delta(T_k)) \geq 3t + 1.$$

This generalises to the CVRP context by the observation that routes must enter and leave both the handle and each tooth multiple times.

2.4.1 Families of Valid Inequalities IX

Comb Inequality: Handle H and Teeth T_1, T_2, T_3

H (handle)



$$\text{Comb inequality: } x(\delta(H)) + \sum_{t=1}^{|T|} x(\delta(T_t)) \geq 3|T| + 1$$

Comb structure with handle H (blue ellipse) and three teeth T_1, T_2, T_3 (orange ellipses). Each tooth partially overlaps with the handle. The comb inequality requires:

$$x(\delta(H)) + x(\delta(T_1)) + x(\delta(T_2)) + x(\delta(T_3)) \geq 3 \times 3 + 1$$

Fractional LP solutions frequently violate this bound, and adding it typically raises the LP lower bound by 2–5%.

2.4.1 Families of Valid Inequalities X

Key Insight: Which Cuts Are Most Effective?

Computational experiments by Lysgaard, Letchford, and Eglese (2004) on benchmark CVRP instances (E-instances of Christofides and Eilon, 1969) show that:

- 1 **RCI/GCI** are always applied first; they are cheap to separate (polynomial-time exact separation via minimum cut).
- 2 **Framed capacity inequalities** provide the biggest improvement in LP bound per separation call.
- 3 **Multistar and hypotour** inequalities are very effective but require more expensive separation.
- 4 **Comb** and other TSP-type inequalities are used for the hardest instances.

Using all four families, the B&C of Lysgaard et al. (2004) solves E-instances up to $n = 135$ customers routinely and set records for many benchmark instances.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures**
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography

2.4.2 Separation Procedures I

A *separation procedure* takes the current LP solution x^* and either (a) finds a valid inequality violated by x^* (i.e., a *cut*), or (b) certifies that no such inequality exists in the chosen family. The efficiency and quality of separation procedures is crucial: if separation is too slow, the B&C overhead outweighs the benefit of cuts.

Separation for RCI/GCI. Finding the most violated RCI requires solving a *minimum* (s, t) -*cut* problem: choose S to minimise $x(\delta(S)) - 2r(S)$. This is done as follows:

- 1 Build a capacitated graph where arc (i, j) has capacity x_{ij}^* .
- 2 For each customer i , run a minimum cut between i and the depot 0. The minimum cut gives the subset S minimising $x^*(\delta(S))$; check if $x^*(\delta(S)) < 2r(S)$.
- 3 Use Gomory-Hu trees (Padberg and Rinaldi, 1990) to compute all pairwise minimum cuts efficiently in $O(n)$ max-flow calls.

2.4.2 Separation Procedures II

Separation for Framed Capacity Inequalities. The separation procedure for FCI, described by Toth and Vigo (2002), solves a series of minimum-cut sub-problems with modified capacities to find the subset S and frame F yielding the most violated FCI. While more expensive than RCI separation, it is still polynomial.

Separation for Comb Inequalities. Exact separation of all comb inequalities is NP-hard in general. In practice, heuristic procedures are used:

- **Template-based search:** identify candidate handles from the fractional solution graph (highly fractional connected components).
- **Shrinking:** merge vertices with x_{ij}^* close to 1, reducing the graph before searching for combs.
- **Sequential lifting:** start with a violated RCI as handle; find teeth that maximise violation.

These heuristics are not guaranteed to find all violated combs but work very well on benchmark instances.

2.4.2 Separation Procedures III

Separation for Multistar Inequalities. Lysgaard et al. (2004) propose an exact polynomial separation for multistar inequalities based on the observation that the most violated multistar corresponds to maximising a quadratic function over a neighbourhood of the current LP solution. This can be solved efficiently using a sequence of minimum-cut computations.

Summary of Separation Complexity

Inequality family	Separation complexity	Reference
RCI / GCI	Polynomial (min-cut)	Padberg & Rinaldi 1991
Framed capacity	Polynomial (min-cut)	Toth & Vigo 2002
Hypotour	Polynomial (LP)	Gendreau et al. 1995
Multistar	Polynomial	Lysgaard et al. 2004
Comb	NP-hard (heuristic)	Padberg & Rinaldi 1991

2.4.2 Separation Procedures IV

Key Insight: The Cutting Plane Cycle

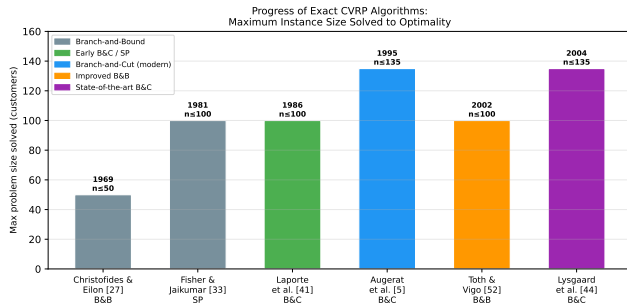
In B&C, at each node one iterates the *cutting plane cycle*: solve LP $\rightarrow$ separate $\rightarrow$ add cuts $\rightarrow$ re-solve LP $\rightarrow$... until either (a) no violated inequality is found, (b) the LP solution becomes integer, or (c) a limit on the number of cuts per node is reached. The cycle typically terminates after 5–20 rounds at the root node, raising the LP bound by 10–30% compared to the bare LP relaxation. At other nodes, reusing cuts from the parent accelerates convergence.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods**
- 17 Conclusions and Future Directions
- 18 Bibliography

2.4.3 Comparison of Branch-and-Cut Algorithms I

Several B&C algorithms have been proposed for the CVRP since the mid-1980s, progressively solving larger instances. We summarise the landmark algorithms and their key characteristics.



The figure shows the maximum CVRP instance size solved to proven optimality by each major algorithm, from 1969 to 2004. Progress has been roughly linear on a log scale: roughly doubling the instance size every decade, driven by better cutting planes and faster hardware.

2.4.3 Comparison of Branch-and-Cut Algorithms II

Key Exact Algorithms: A Timeline

Year	Authors	Key contribution
1969	Christofides & Eilon	First B&B with AP bound; $n \leq 50$
1981	Fisher & Jaikumar	SP with generalised assignment; $n \leq 100$
1986	Laporte et al.	B&C with RCI; $n \leq 100$
1989	Agarwal et al.	SP column generation; $n \leq 45$
1995	Augerat et al.	B&C with GCI + multistar; $n \leq 135$
2002	Toth & Vigo	Improved B&B; $n \leq 100$
2004	Lysgaard et al.	CVRPSEP: full cut library; $n \leq 135+$

2.4.3 Comparison of Branch-and-Cut Algorithms III

The CVRPSEP library. Lysgaard, Letchford, and Eglese (2004) developed CVRPSEP, a publicly available library implementing separation routines for all major CVRP valid inequalities. Their B&C algorithm uses:

- **LP relaxation** solved by CPLEX at every node.
- **Cutting plane cycle** with RCI, multistar, hypotour, and comb separation.
- **Branching** on the most fractional arc variable.
- **Best-first** node selection with occasional depth-first dives.

On the standard E-instances of Christofides and Eilon (sizes 22 to 101), they solved all instances to optimality in under 100 seconds on a 1.4 GHz machine. On larger CMT instances (Christofides, Mingozzi, Toth), they solved all instances up to $n = 199$ within reasonable time limits, setting new records.

2.4.3 Comparison of Branch-and-Cut Algorithms IV

The algorithm of Naddef and Rinaldi (2002). Working independently, Naddef and Rinaldi developed a B&C based on the symmetric TSP formulation with additional capacity cuts. They used a fractional relaxation combined with cutting planes derived from the polyhedral theory of the symmetric TSP. Their algorithm solved the 135-customer instance “E101-k8” to optimality, which had been open for many years.

The algorithm of Baldacci, Christofides, and Mingozzi (2008). A subsequent breakthrough was achieved by using an entirely different bounding approach: the *state-space relaxation* combined with ng-route relaxations in a column-generation framework. This algorithm, not a pure B&C, proved optimality for all standard benchmark instances up to $n = 200$ customers.

2.4.3 Comparison of Branch-and-Cut Algorithms V

Example: Solving an E-instance

Consider instance E-n22-k4 (22 customers, 4 vehicles, from Christofides and Eilon, 1969). The LP relaxation at the root node gives a lower bound of, say, 373.4. The heuristic upper bound is 381. After the cutting-plane cycle at the root:

- 1 8 RCI cuts raise LB to 376.2.
- 2 3 framed capacity cuts raise LB to 378.5.
- 3 1 comb cut raises LB to 380.1.

Now $LB=380.1$ and $UB=381$, giving a gap of only 0.24%. B&C branches once, the child node LP gives 381.0, and the integer solution is confirmed as optimal. Total: 1 LP at root + 8 cuts + 1 branch = very fast.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions**
- 18 Bibliography

2.5 Conclusions and Future Research Directions I

This chapter has reviewed the main exact algorithmic approaches for the CVRP, from the early B&B methods of the late 1960s to the state-of-the-art B&C algorithms of the 2000s. Several key conclusions emerge.

Progress and current state. The progress in exact CVRP algorithms has been substantial: from $n \leq 50$ in 1969 to $n \leq 199$ in 2008, a fourfold increase in problem size. This progress has been driven primarily by:

- 1 Better formulations (arc-variable, set partitioning).
- 2 Tighter lower bounds (LP relaxations, column generation).
- 3 Richer families of valid inequalities (RCI, comb, multistar).
- 4 Faster computers (Moore's law).

2.5 Conclusions and Future Research Directions II

Remaining challenges. Despite this progress, several open problems remain:

- **Larger instances:** instances with $n > 200$ customers remain very challenging, even for the best exact algorithms.
- **Stronger lower bounds:** the gap between the LP relaxation and the integer optimum is sometimes 5–15% for large instances, suggesting many cutting planes are still missing from current libraries.
- **Asymmetric CVRP:** extensions to directed arcs and asymmetric cost matrices are harder; less progress has been made.
- **CVRP variants:** exact algorithms for CVRPTW, multi-depot CVRP, and other variants are less developed.

2.5 Conclusions and Future Research Directions III

Future research directions. The authors identify several promising directions:

- ① **New valid inequalities:** deriving deeper cuts from the polyhedral structure of the CVRP, possibly combining capacity and routing constraints.
- ② **Improved pricing subproblems:** more efficient algorithms for the ESPPRC, the bottleneck in column-generation-based B&C.
- ③ **Parallel B&C:** exploiting multi-core and cluster computing to explore multiple nodes simultaneously.
- ④ **Hybrid methods:** combining exact B&C with metaheuristics (matheuristics) to provide tight bounds for instances beyond the reach of pure exact methods.
- ⑤ **Machine learning:** using ML to learn branching strategies, cut selection policies, and pricing heuristics, as has been done for general MIP solvers.

2.5 Conclusions and Future Research Directions IV

Overall Takeaway

The CVRP is one of the most intensively studied combinatorial optimisation problems, and the development of exact algorithms has produced a rich body of mathematical theory (polyhedral theory, column generation, cutting planes) with wide applicability beyond routing. The Branch-and-Cut paradigm, enriched with a comprehensive library of valid inequalities and efficient separation routines, remains the state of the art for proving CVRP optimality. Future advances will likely come from a combination of stronger mathematical structure, better algorithms for subproblems, and the integration of machine learning techniques.

Section Outline

- 1 Introduction: Why Exact Methods Matter
- 2 Branch-and-Bound Algorithms
- 3 Early Set Partitioning Algorithms
- 4 Branch-and-Cut Algorithms
- 5 Families of Cuts
- 6 Separation Procedures
- 7 Comparison of B&C Methods
- 8 Conclusions and Future Directions
- 9 Bibliography
- 10 Introduction: Why Exact Methods Matter
- 11 Branch-and-Bound Algorithms
- 12 Early Set Partitioning Algorithms
- 13 Branch-and-Cut Algorithms
- 14 Families of Cuts
- 15 Separation Procedures
- 16 Comparison of B&C Methods
- 17 Conclusions and Future Directions
- 18 Bibliography**

Selected Bibliography I

The following references are cited in Chapter 2. They are organised chronologically to illustrate the progression of the field.

-  P. Christofides and S. Eilon, "An algorithm for the vehicle-dispatching problem," *Operational Research Quarterly*, 20 (1969), pp. 309–318.
-  M.L. Fisher and R. Jaikumar, "A generalised assignment heuristic for vehicle routing," *Networks*, 11 (1981), pp. 109–124.
-  N. Christofides, A. Mingozzi, and P. Toth, "Exact algorithms for the vehicle routing problem, based on spanning tree and shortest path relaxations," *Mathematical Programming*, 20 (1981), pp. 255–282.
-  G. Laporte, H. Mercure, and Y. Nobert, "An exact algorithm for the asymmetrical capacitated vehicle routing problem," *Networks*, 16 (1986), pp. 33–46.
-  Y. Agarwal, K. Mathur, and H.M. Salkin, "A set-partitioning-based exact algorithm for the vehicle routing problem," *Networks*, 19 (1989), pp. 731–749.

Selected Bibliography II

-  P. Augerat, J.M. Belenguer, E. Benavent, A. Corberán, D. Naddef, and G. Rinaldi, “Computational results with a branch-and-cut code for the capacitated vehicle routing problem,” Research Report 949-M, 1995.
-  M. Gendreau, G. Laporte, and R. Seguin, “A tabu search heuristic for the vehicle routing problem,” *Management Science*, 40 (1994), pp. 1276–1290.
-  P. Toth and D. Vigo, “Models, relaxations and exact approaches for the capacitated vehicle routing problem,” *Discrete Applied Mathematics*, 123 (2002), pp. 487–512.
-  J. Lysgaard, A.N. Letchford, and R.W. Eglese, “A new branch-and-cut algorithm for the capacitated vehicle routing problem,” *Mathematical Programming*, 100 (2004), pp. 423–445.
-  D. Naddef and G. Rinaldi, “Branch-and-cut algorithms for the capacitated VRP,” in *The Vehicle Routing Problem*, P. Toth and D. Vigo, eds., SIAM, 2002, pp. 53–84.
-  R. Baldacci, N. Christofides, and A. Mingozzi, “An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts,” *Mathematical Programming*, 115 (2008), pp. 351–385.

Selected Bibliography III



R. Baldacci, A. Mingozzi, and R. Roberti, “New route relaxation and pricing strategies for the vehicle routing problem,” *Operations Research*, 59 (2011), pp. 1269–1283.

Appendix A: CVRP Data Structures in Python

The following Python code defines the basic data structures for representing a CVRP instance and computing the objective value of a solution.

Listing 1: CVRP instance and solution evaluation

```
import numpy as np
from dataclasses import dataclass, field
from typing import List, Dict

@dataclass
class CVRPInstance:
    """
    A CVRP instance.
    n          : number of customers (vertices 1..n; 0 = depot)
    coords     : (n+1) x 2 array, coords[0] = depot
    demands    : length-(n+1) array; demands[0] = 0
    capacity   : vehicle capacity Q
    """
    n: int
    coords: np.ndarray # shape (n+1, 2)
    demands: np.ndarray # shape (n+1,)
    capacity: float

    def cost_matrix(self) -> np.ndarray:
        """Euclidean distance matrix (n+1) x (n+1)."""
        diff = self.coords[:, None, :] - self.coords[None, :, :]
        return np.sqrt((diff**2).sum(axis=2))
```

Appendix B: Greedy Nearest-Neighbour Heuristic (Upper Bound)

Before running exact B&C, a quick heuristic is needed to provide an initial upper bound. The nearest-neighbour heuristic greedily constructs routes by always visiting the nearest unvisited customer.

Listing 2: Nearest-neighbour heuristic for CVRP

```
def nearest_neighbour_cvrp(inst: CVRPInstance
                           ) -> List[List[int]]:
    """
    Greedy nearest-neighbour CVRP heuristic.
    Returns a list of routes (each route = list of customer indices).
    """
    C = inst.cost_matrix()
    unvisited = set(range(1, inst.n + 1))
    routes = []

    while unvisited:
        route = []
        load = 0.0
        current = 0  # start at depot

        while unvisited:
            # Find nearest feasible unvisited customer
            best = None
            best_dist = float('inf')
            for j in unvisited:
                if (load + inst.demands[j] <= inst.capacity
                    and C[current, j] < best_dist):
```

Appendix C: Rounded Capacity Inequality Separation

The following code implements a simple exact separation procedure for the Rounded Capacity Inequality (RCI) using a minimum-cut approach.

Listing 3: RCI separation using minimum cut (simplified)

```
import networkx as nx
import math
from itertools import combinations

def separate_rci(inst: CVRPInstance,
                 x_frac: Dict) -> List[tuple]:
    """
    Separate Rounded Capacity Inequalities.
    x_frac: dict {(i,j): x*_{ij}} fractional LP solution.
    Returns list of violated RCIs as (S, lhs, rhs) tuples.
    """
    violations = []
    n = inst.n

    # Build a graph with LP solution values as capacities
    G = nx.Graph()
    for (i, j), val in x_frac.items():
        if val > 1e-6:
            G.add_edge(i, j, capacity=val)

    # For each customer k, find min cut between k and depot 0
    for k in range(1, n + 1):
```

Appendix D: Branch-and-Bound Skeleton

The following code provides a skeleton implementation of a simple Branch-and-Bound algorithm for CVRP, illustrating the key components discussed in the chapter. This is a conceptual implementation; a production solver would use an LP solver (e.g., CPLEX or Gurobi) and a much richer set of cutting planes.

Listing 4: Branch-and-Bound skeleton for CVRP

```
from heapq import heappush, heappop

def branch_and_bound_cvrp(inst: CVRPInstance,
                          compute_lb,      # callable: node -> lb
                          get_heuristic    # callable: inst -> routes
                          ):
    """
    Minimalist B&B skeleton for CVRP.
    compute_lb(node)    -> float: lower bound for node subproblem
    get_heuristic(inst)-> routes: initial UB
    """
    # Initialise
    init_routes = get_heuristic(inst)
    UB = solution_cost(inst, init_routes)
    best_solution = init_routes

    # Priority queue: (lb, node_id, node_data)
    node_id = 0
    root = {'fixed_1': [], 'fixed_0': []} # fixed arcs
    open_list = []
    heappush(open_list, (0.0, node_id, root))
```

Appendix E: Valid Inequalities — Summary Table I

This table summarises all the valid inequality families discussed in the chapter, together with their key properties.

Name		Form	Parameters	Separation
Rounded (RCI)	Capacity	$x(\delta(S)) \geq 2\lceil d(S)/Q \rceil$	$S \subseteq V'$	Poly (min-cut)
Generalised (GCI)	Cap.	$x(\delta(S)) \geq 2\lceil d(S)/Q \rceil$	$S \subseteq V'$	Poly (min-cut)
Framed (FCI)	Capacity	$x(\delta(S)) \geq 2r(S) + 2r(F \setminus S) - r(F)$	$S \subseteq F \subseteq V'$	Poly
Hypotour		$x(E(S)) \leq S - r(S)$	$S \subseteq V'$	Poly (LP)
Multistar		$x(\delta(S)) \geq 2r(S) + \max_v[\dots]$	$S \subseteq V'$	Poly
Comb		$x(\delta(H)) + \sum_k x(\delta(T_k)) \geq 3t + 1$	Handle H , Teeth T_k	NP-hard (heuristic)

Appendix E: Valid Inequalities — Summary Table II

How to read the table. For each inequality family, the “Form” column gives the mathematical statement of the inequality, where S (and F , H , T_k) are the subsets of customers that parametrise the family. The “Separation” column indicates the computational complexity of finding the most violated inequality in the family given a fractional LP solution. “Poly” means a polynomial-time exact algorithm exists; “NP-hard (heuristic)” means exact separation is NP-hard, and heuristic procedures are used in practice.